

typical terminal output.

So far, so good... Now about compilation:

```
fakeroot make-kpkg --initrd --append-to-version=-custom kernel_
image kernel_headers
```

This command is most likely to work. If anything goes wrong, just delete the entry in GRUB and give another try after logging into the system using the old kernel version.

Logical filesystems

It is quite comfortable to access block devices with a logical filesystem. This can be mounted in the virtual filesystem; which implies that the block device contains file (and structure) details, which permit the logical filesystem to access the device. You need to note that the device can only support one logical filesystem at a time.

You can guess that when a filesystem is made to mount as a subdirectory, the directories and files existing on the device will be seen as subdirectories of the mount point (which itself is a subdirectory). This also offers a great amount of abstraction (as the users need not know the specific details of the logical filesystem) and flexibility.

You might already know that Linux uses the concept of *inodes* to represent a file on a block device and one can also comprehend that the idea is virtual. It can be used as a storage location for any information concerning an open file on the disk, and it stores associated buffers (and even the mapping between device blocks and file offsets).

Linux kernel limits

Here is an overview of the default Linux kernel limits (the kernel parameter):

1. Maximum number of processes

```
NR_TASKS 512
```

```
MAX_TASKS_PER_USER (NR_TASKS/2)
```

2. Semaphores

```
SEMMNI 128 /* ? max # of semaphore identifiers */
```

```
SEMMSL 32 /* <= 512 max num of semaphores per id */
```

```
SEMMNS (SEMMNI*SEMSL) /* ? max # of semaphores in system
```

```
*/
SEMOPM 32 /* ~ 100 max num of ops per semop call */
SEMMVMX 32767 /* semaphore maximum value */
```

```
/* unused */
SEMUME SEMOPM /* max num of undo entries per process */
SEMMNU SEMMNS /* num of undo structures system wide */
SEMAEM (SEMMVMX >> 1) /* adjust on exit max value */
SEMMAP SEMMNS /* # of entries in semaphore map */
SEMSUZ 20 /* size of struct sem_undo */
```

3. Shared memory

```
/*
 * Keep _SHM_ID_BITS as low as possible since SHMMNI
 * depends on it and there is a static array of size SHMMNI.
 */
_SHM_ID_BITS 7
_SHM_IDX_BITS 15

/*
 * _SHM_ID_BITS + _SHM_IDX_BITS must be <= 24 on the i386 and
 * SHMMAX <= (PAGE_SIZE << _SHM_IDX_BITS).
 */
SHMMAX 0x2000000 /* max shared seg size (bytes) */
SHMMIN 1 /* really PAGE_SIZE */ /* min shared seg size (bytes) */
SHMMNI (1<<_SHM_ID_BITS) /* max num of segs system wide */
SHMALL (1<<(_SHM_IDX_BITS+_SHM_ID_BITS))
/* max shm system wide (pages) */
SHMLBA PAGE_SIZE /* attach addr a multiple of this */
SHMSEG SHMMNI /* max shared segs per process */
```

4. Number of open files

```
NR_INODE 3072 /* this should be bigger than NR_FILE */
```

```
NR_FILE 1024 /* this can well be larger on a larger system */
```

Now, it is time to wind up this voyage. IPC, external interface and data structures are some of the important topics that we will discuss next month. Experiment with your kernel and let me know the results.

Happy kernel hacking! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com